



Програмиране за мобилни интернет устройства

УВОД. ЕЗИЦИ ЗА ПРОГРАМИРАНЕ И КОНЦЕПЦИИ В
АНДРОИД. КОТЛИН. ОСНОВНИ КОНЦЕПЦИИ НА
ПРОГРАМНИЯ ЕЗИК

История на Андроид

- ▶ Операционна система предназначена предимно за мобилни устройства, базирана на „линукс ядро“
- ▶ 2005 - Android Inc. закупени от Google. Начало на разработката на Далвийк виртуалната машина;
- ▶ 2007 - Open Handset Alliance (Samsung, Motorola, Sony Ericson, T-Mobile, Toshiba, Vodafone, Google, Intel, Texas Instruments)
- ▶ 2008 - Отваряне на кода на Андроид под лиценз на Apache
 - ▶ T-Mobile G1 телефон – Андроид 1.0, Android Dev Phone 1
 - ▶ липса на виртуална клавиатура (ограничен набор от устройства)
- ▶ 2009 - SDK 1.5 , Андроид 1.6, 2.0



История на Андроид

- ▶ 2008 – 1.0
- ▶ 9 Февруари 2009 - 1.1
- ▶ 30 Април 2009 1.5 Cupcake
- ▶ 15 Септември 2009 1.6 Donut
- ▶ 26 Октомври 2009 2.0/2.1 Eclair
- ▶ 20 Май 2010 2.2 Froyo
- ▶ 6 Декември 2010 2.3 Gingerbread
- ▶ 22 Февруари 2011 3.0 Honeycomb
- ▶ 19 Октомври 2012 4.0 Ice Cream Sandwich
- ▶ 9 Юли 4.1 Jelly Bean
- ▶ 31 Октомври 2013 4.4 KitKat
- ▶ 3 Ноември 2014 5.0 LooliPop
- ▶ 5 Октомври 2015 6.0 Marshmallow
- ▶ 22 Август 2016 7.0 Nougat
- ▶ 21 Август 2017 8.0 Oreo
- ▶ 6 Август 2018 9.0 Pie
- ▶ 3 Септември 2019 10
- ▶ 8 Септември 2020 11
- ▶ 12 Октомври 2021 12
- ▶ 15 Август 2022 13
- ▶ 7 Август 2023 14

РАЗВИТИЕ

- ▶ Gingerbread (2.3.3, 2.3.4)
 - ▶ Bluetooth
 - ▶ Media framework
 - ▶ Speech recognition
 - ▶ подобрение на графични интерфейс
- ▶ Honeycomb (3.0, 3.1, 3.2)
 - ▶ специална оптимизация за устройства с голям екран
 - ▶ 3D десктоп с нови уиджети
 - ▶ Преработен многозадачен механизъм
 - ▶ Нови възможности на уеб браузъра – табове, автоматично попълване на форми, синхронизиране на отметки, private сърфиране и др.
 - ▶ Поддръжка на много-процесорни архитектури.

РАЗВИТИЕ

► Ice Cream Sandwich (4.0)

- Подобрения на графични интерфейс, оптимизация за екрани с висока резолюция
- Подобрен многозадачен режим на работа
- Преоразмеряващи се уиджети
- Нови действия за заключване
- Бърз отговор на повиквания
- Подобрение на въвеждането на текст и проверката на правописа и др.

► Jelly Bean (4.1, 4.2, 4.3)

- Vsync timing
- Triple buffering
- Редуциране на латентността при работа с екрана
- CPU input boost
- Хардуерно ускорение на 2D рендирането
- Многозадачност при многоядрена архитектура
- OpenGL ES 3.0
- Bluetooth Smart технология и др.

Развитие

- ▶ KitKat (4.4)
 - ▶ Оптимизации по управление на паметта
 - ▶ Near field communication (NFC) – транзакции с Host Card Emulation
 - ▶ Фреймуърк за принтиране
 - ▶ Нови механизми на запазване на данни
 - ▶ Сензори с ниска консумация
 - ▶ Анализ на поведението на приложенията и др.

Развитие

- ▶ Lollipop (5.0, 5.1)
 - ▶ Нов подход при изграждане на графичния интерфейс - Material design
 - ▶ Конкурентен достъп до ресурси от екрана
 - ▶ Подобрение на WebView
 - ▶ Снимка на екрана и споделяне
 - ▶ Съобщения при заключен екран
 - ▶ Допълнителни данни за съобщенията
 - ▶ Поддръжка на OpenGL ES 3.1

Lollipop 5.x

- ▶ Подобрение по мултимедийните елементи – камера, аудио
 - ▶ Достъп до мултимедийно съдържание на друго приложение
- ▶ Подобрение по съхранението на данни – избор на директория и гарантиран постоянен достъп до нея
- ▶ Подобрение на мрежовата поддръжка – множество и различни мрежи по едно и също време
- ▶ Bluetooth с ниска консумация
- ▶ Подобрение на NFC
- ▶ Подобрение по отношението на управлението на консумация на енергия – проект Volta

Lollipop 5.x

- ▶ Android Runtime (ART)
 - ▶ Ahead-of-time (AOT) компилация
 - ▶ Подобрен garbage collector
 - ▶ Подобрен debug, инструменти за това
 - ▶ *Ниска толерантност към програмите*
- ▶ Поддръжка на 64-Bit в Android NDK

Marshmallow 6.x

- ▶ Marshmallow(API 23)
 - ▶ Контекстно търсене по ключови думи в рамките на приложения
 - ▶ режим Doze, намаляване скоростта на процесора, докато екранът е изключен
 - ▶ Режим на готовност на приложенията
 - ▶ Работа с пръстови отпечатъци
 - ▶ Директен режим на споделяне
 - ▶ Динамични права за достъп след режим на инсталация
 - ▶ Автоматично архивиране и възобновяване на системата
 - ▶ Адаптивна външна памет
 - ▶ Многопрозоръчен режим
 - ▶ Поддръжка на MIDI интерфейс

Marshmallow 6.x

- ▶ Спиране на поддръжката на Apache HTTP клиент
 - ▶ HttpURLConnection
- ▶ Замяна на OpenSSL с BoringSSL
- ▶ Фактори дизайн шаблон за нотификации
- ▶ Промяна в управлението на достъпа до звук
- ▶ Промяна в организацията на мрежови връзки
- ▶ Стриктна валидация на APK файловете
- ▶ Промяна в политиката на използване на USB порта

Nougat 7.x

- ▶ Nougat (API 24,25)
 - ▶ калибриране на цветовете
 - ▶ Промяна в начина на актуализация на системата (втори системен дял)
 - ▶ VR interface
 - ▶ Нови компилатори на виртуалната машина
 - ▶ Подобрения по интерфейс и различни базови функционалности

Oreo 8.x

- ▶ Oreo (API level 26)
- ▶ Напълно нов подход във всяко ниво на операционната система
- ▶ Подобрения – безброй 😊
 - ▶ Нови изисквания към приложенията
 - ▶ Много методи от предни API са deprecated
 - ▶ Задължителност за използване на новите имплементации
 - ▶ Промяна на организацията на работа на услугите
 - ▶ Използване на *JobIdManager*
 - ▶ Реорганизация на основни процеси в работата на ядрото
 - ▶ Подобряване на сигурността



Pie 9

- ▶ Нови възможности за взаимодействие с потребителите - hybrid gesture/button navigation system
- ▶ Интелигентен отговор навсякъде
- ▶ Интелигентен избор на текст в интерфейса за преглед
- ▶ Избор на изображение в интерфейса за преглед
- ▶ Срезове — части от приложения в търсене
- ▶ По-лесно управление на екранни снимки
- ▶ По-интелигентно управление на батерията
- ▶ По-интелигентен контрол на екрана
- ▶ По-малко досадно завъртане на екрана
- ▶ По-лесен начин за регулиране на шума

10

- ▶ Край на захарната болест
- ▶ Нов интерфейс за Android gestures
- ▶ Нова система за достъп до ресурси



- ▶ Силна еволюция на 10
- ▶ Засилване на сигурността и поверителността
 - ▶ Системата за разрешения
 - ▶ Автоматична отмяна
- ▶ Нов подход за нотификации
- ▶ Ново приложение за потоково видео
- ▶ Native запис на екран
- ▶ Меню на системно ниво за свързани устройства

12,13,14

- ▶ Нов интерфейс
- ▶ Нов интерфейс за съвдаеми телефони и таблети
 - ▶ Chrome подобен интерфейс
 - ▶ Нова система за паралелна обработка
- ▶ Поддръжка на клониране на приложения и едновременна работа на инстанциите му

Андроид устройства

- ▶ Устройства
 - ▶ Смартфони
 - ▶ Таблети
 - ▶ Електронни четци
 - ▶ Netbooks
 - ▶ MP4 плеари
 - ▶ Интернет телевизори
- ▶ голямо разнообразие на устройства - тип/форма, версии на ОС, екрани (резолюция и плътност)

ИЗИСКВАНИЯ

- ▶ ограничени възможности по отношение на консумация на енергия, изчислителна мощ и количество памет
- ▶ необходимост от оптимизиране на всички технологични процеси за да отговорят на изискванията
- ▶ преработване на стандартната JVM – Dalvik VM, два и повече пъти намаляване на размера на файловете (паметта)
- ▶ Java Native Interface (C, C++)

Инструменти

- ▶ JDK;
- ▶ Android SDK;
- ▶ Eclipse IDE;
 - ▶ Android Development Tools (ADT)
- ▶ IntelliJ Idea
- ▶ Android studio
- ▶ Gradle (Gradle plugin)
 - ▶ Зависимости между програмни компоненти
 - ▶ Хранилища за библиотеки
 - ▶ Процес на изграждане на приложения

Android SDK

- ▶ API Android SDK
- ▶ Документация
- ▶ Android Virtual Device – емуляция на андроид устройство
- ▶ Development Tools – инструментални средства за компилиране и внедряване на програмите
- ▶ Sample Code

Версии SDK и Android API Level

- ▶ Android 1.0 (API level 1)
- ▶ Android 1.1 (API level 2)
- ▶ Android 1.5 Cupcake (API level 3)
- ▶ Android 1.6 Donut (API level 4)
- ▶ Android 2.0 Eclair (API level 5)
- ▶ Android 2.0.1 Eclair (API level 6)
- ▶ Android 2.1 Eclair (API level 7)
- ▶ Android 2.2–2.2.3 Froyo (API level 8)
- ▶ Android 2.3–2.3.2 Gingerbread (API level 9)
- ▶ Android 2.3.3–2.3.7 Gingerbread (API level 10)
- ▶ Android 3.0 Honeycomb (API level 11)
- ▶ Android 3.1 Honeycomb (API level 12)
- ▶ Android 3.2 Honeycomb (API level 13)
- ▶ Android 4.0–4.0.2 Ice Cream Sandwich (API level 14)
- ▶ Android 4.0.3–4.0.4 Ice Cream Sandwich (API level 15)
- ▶ Android 4.0.3–4.0.4 Ice Cream Sandwich (API level 15)
- ▶ Android 4.1 Jelly Bean (API level 16)
- ▶ Android 4.2 Jelly Bean (API level 17)
- ▶ Android 4.3 Jelly Bean (API level 18)
- ▶ Android 4.4 KitKat (API level 19)
- ▶ Android Lollipop 5.0-5.1 (API level 20-22)
- ▶ Android Marshmallow 6.0 (API level 23)
- ▶ Android Nougat 7.0 (API level 24-25)
- ▶ Android Oreo 8.0 (API Level 26)
- ▶ Android 8.1 (API level 27)
- ▶ Android 9 (API level 28)
- ▶ Android 10 (API level 29)
- ▶ Android 11 (API level 30)
- ▶ Android 12 (API level 31,32)
- ▶ Android 13 (API Level 33)
- ▶ Android 14 (API Level 34)

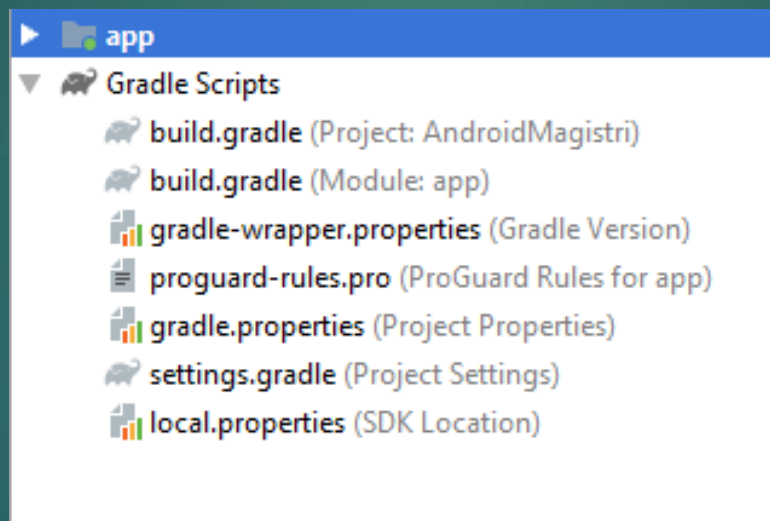
Инструменти

- ▶ Езици за програмиране
 - ▶ Java
 - ▶ Kotlin
 - ▶ 2011
 - ▶ Project Kotlin
 - ▶ 2016 – обявен като независим език за програмиране

Инструмент за създаване и управление на Android проект

- ▶ Gradle
 - ▶ Самостоятелна система за създаване компилиране и управление
 - ▶ Андроид студио - плъгин
 - ▶ JVM инструмент
 - ▶ Java, Groovy
 - ▶ Scala
 - ▶ Компиляция при промяна
 - ▶ Многокомпонентна поддръжка
 - ▶ Йерархия от скриптове
 - ▶ Нива по модули

Компилиране на проект и изпълнение. Gradle.



Въведение в Котлин

- ▶ JVM език
 - ▶ Свой компилатор и среда за изпълнение
 - ▶ По-лесен синтаксис
 - ▶ По-бързо изпълнение
 - ▶ По-добра поддръжка на функции и ламбда изрази
 - ▶ По-добра поддръжка на корутини и асинхронно програмиране
- ▶ Функционална Java

Променливи и типове данни

- ▶ Kotlin поддържа различни типове променливи:
- ▶ `var`: Използва се за променливи, които могат да бъдат променяни (mutable).
- ▶ `val`: Използва се за константи (immutable).
- ▶ Пример:
 - ▶ `var age: Int = 30`
 - ▶ `val name: String = "Ivan"`

ПРОМЕНЛИВИ И ТИПОВЕ ДАНИИ

- ▶ Типове данни в Kotlin:
 - ▶ Цели числа: Byte, Short, Int, Long
 - ▶ Дробни числа: Float, Double
 - ▶ Логически: Boolean
 - ▶ Текстови: Char, String
- ▶ Пример:
 - ▶ `val isActive: Boolean = true`
 - ▶ `val temperature: Float = 36.6f`

Условни оператори

- ▶ Условните оператори в Kotlin включват:
 - ▶ if, else if, else: Използват се за изпълнение на различни блокове от код в зависимост от условията.

- ▶ Пример:

```
val score = 85
if (score >= 90) {
    println("Отличен")
} else if (score >= 75) {
    println("Много добър")
} else {
    println("Добър")
}
```

Условни оператори

- ▶ when: - Подобно на switch в други езици

- ▶ Пример:

```
val grade = 'A'
when (grade) {
    'A' -> println("Отличен")
    'B' -> println("Много добър")
    'C' -> println("Добър")
    else -> println("Неуспешен")
}
```

ЦИКЛИ

- ▶ Kotlin поддържа няколко вида цикли:
 - ▶ `for`: Използва се за итериране през колекции и диапазони.
- ▶ Пример:

```
for (i in 1..5) {  
    println(i)  
}
```

ЦИКЛИ

- ▶ `while`: Изпълнява блок от код, докато условието е вярно.
- ▶ `do-while`: Изпълнява блок от код поне веднъж, след което продължава, докато условието е вярно.

- ▶ Пример:

```
var i = 5
while (i > 0) {
    println(i)
    i--
}
```


ЦИКЛИ

- ▶ Пример за do-while:

```
var j = 5
```

```
do {
```

```
    println(j)
```

```
    j--
```

```
} while (j > 0)
```

ФУНКЦИИ

- ▶ Функциите са основни градивни блокове в Kotlin
 - ▶ **Глобални функции:** Функции, които са дефинирани директно в пакет, без да са част от клас или обект. Те могат да бъдат достъпвани от всяка точка на кода чрез директно извикване или чрез импорт.
 - ▶ **Член-функции (методи):** Функции, които са дефинирани вътре в класове или обекти и са свързани с техните инстанции.
 - ▶ **Разширителни функции:** Специален вид функции, които позволяват добавяне на нови методи към съществуващи класове, без да променят техния код.

Глобална функция

// Файл: Utils.kt

```
package com.example.utils
```

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}
```

// Достъп до функцията от друг файл в същия проект

```
import com.example.utils.sum
```

```
fun main() {  
    val result = sum(3, 5)  
    println(result) // Output: 8  
}
```

Член функция (метод)

```
class Calculator {  
    fun multiply(a: Int, b: Int): Int {  
        return a * b  
    }  
}  
  
fun main() {  
    val calculator = Calculator()  
    val product = calculator.multiply(3, 5)  
    println(product) // Output: 15  
}
```

Разширителна функция

```
fun String.removeFirstLastChar(): String {  
    return this.substring(1, this.length - 1)  
}
```

```
fun main() {  
    val str = "Hello"  
    println(str.removeFirstLastChar()) // Output: ell  
}
```

ФУНКЦИИ

- ▶ Функции с връщане на стойност:

- ▶ Пример:

```
fun add(a: Int, b: Int): Int {  
    return a + b  
}
```

- ▶ Викане на функция и използване на върната стойност:

- ▶ `val sum = add(3, 5)`
- ▶ `println("Sum: \sum")`

Класове и Обекти

- ▶ Основи на класове и обекти в Kotlin:
- ▶ Пример за клас:
 - ▶ `class Person(val name: String, var age: Int)`
- ▶ Създаване на обект:
 - ▶ `val person = Person("Ivan", 30)`
 - ▶ `println(person.name)`

Класове и Обекти

- ▶ Методи в класове и обекти:

- ▶ Пример за метод:

```
class Person(val name: String, var age: Int) {  
    fun greet() {  
        println("Hello, my name is \"$name\"")  
    }  
}
```

- ▶ Използване на метод:

- ▶ `val person = Person("Ivan", 30)`
- ▶ `person.greet()`

Интерфейси и Абстрактни класове

- ▶ Kotlin поддържа интерфейси и абстрактни класове:

- ▶ Интерфейси:

```
interface Drivable {  
    fun drive()  
}
```

- ▶ Абстрактни класове:

```
abstract class Vehicle {  
    abstract fun start()  
    fun stop() {  
        println("Vehicle stopped")  
    }  
}
```

Интерфейси и Абстрактни класове

► Пример:

```
class Car: Vehicle(), Drivable {  
    override fun start() {  
        println("Car started")  
    }  
  
    override fun drive() {  
        println("Car is driving")  
    }  
}
```

```
val myCar = Car()  
myCar.start()  
myCar.drive()  
myCar.stop()
```

Обработка на ИЗКЛЮЧЕНИЯ

- ▶ Kotlin поддържа обработка на изключения чрез try-catch блокове:
- ▶ Пример:

```
fun divide(a: Int, b: Int): Int {  
    return try {  
        a / b  
    } catch (e: ArithmeticException) {  
        println("Деление на нула")  
        0  
    }  
}
```

- ▶ `val result = divide(10, 0)`
- ▶ `println("Result: \${result}")`

Ламбда функции и ВИСОКО НИВО НА ФУНКЦИИ

- ▶ Ламбда функции:

- ▶ Пример:

```
val sum = { a: Int, b: Int -> a + b }  
println("Sum: \${sum(3, 4)}")
```

- ▶ Високо ниво на функции:

```
fun operate(x: Int, y: Int, operation: (Int, Int) -> Int): Int {  
    return operation(x, y)  
}
```

- ▶ Пример:

```
val result = operate(5, 3, sum)  
println("Result: \${result}")
```